A PROJECT REPORT

ON

UNIVERSITY STUDENT MANAGEMENT SYSTEM

Course: Database Management System Lab

Course Code: 0612-304

SUBMITTED BY:

Md Rayhan Miah

Reg No: CS-D-98-23-127187

Roll: 13

Batch: D-98

Department of Computer Science and Engineering

Dhaka International University

SUBMITTED TO:

Mobarak Hossain Al-Mamon

Lecturer

Department of Computer Science and Engineering

Dhaka International University

Date of Submission: 11 November 2026

Table of Contents

Chapter	Title	Page
1	Introduction	3
2	Project Overview	3
3	ER Model and ER Diagram	4
4	ER-to-Relational Mapping and Table Description	5
5	DDL - Table Management	6
6	DML - Data Manipulation	7
7	Sample Queries and Output	8
8	Conclusion	10

List of Tables: Table 4.1 gives the description of the six tables used in this project. The sample data used in the queries of Chapter 7 is inserted in Chapter 6.

1. Introduction

A university handles records of its departments, teachers, students, courses, enrollments and results. In many traditional educational setups or smaller departments, these records are maintained in paper files or fragmented spreadsheets. Searching an old student record or grade sheet takes time, data gets duplicated, and there is no quick way to determine how many students are enrolled in a specific course, which department a teacher belongs to, or what the overall grade distribution looks like.

In this project, a complete relational database has been designed for a University Student Management System. The database is created and managed entirely through SQL queries in Oracle. There is no external website or graphical software interface in this project; all operations—such as creating tables, inserting records, updating data, and executing complex queries—are performed directly by writing and executing SQL statements in the terminal.

The primary objective of this project is to apply the core concepts taught in the Database Management System Lab course, such as table creation with integrity constraints, data manipulation, filtering, sorting, aggregate functions, grouping, subqueries, and table joins, to a realistic academic management scenario. A secondary objective is to design the database soundly, starting from an Entity-Relationship (ER) model and translating it into well-structured relational tables.

2. Project Overview

The University Student Management System maintains academic and administrative information for multiple departments. Each department employs teachers and admits students. Teachers offer and teach courses assigned by their departments. Students enroll in courses during specific semesters, and their academic performance is tracked through exam results.

The system is capable of answering key administrative questions, such as identifying students in a specific department, finding teachers with salaries above a threshold, calculating total marks and grade point averages, listing enrollments by semester, and analyzing course-wise performance.

2.1 Tables used in the project

The database contains six interconnected tables:

• **Department** - Information about academic departments (ID, name, head, established year).

• **Teacher** - Information about faculty members, their designations, departments, emails, and salaries.

• **Student** - Information about enrolled students, gender, date of birth, phone, and department.

• **Course** - Information about courses offered, credits, department, and assigned teacher.

• **Enrollment** - Records indicating which student is enrolled in which course for a specific semester.

• **Result** - Records of marks and grades obtained by students in their course examinations.

2.2 Tools used

Oracle Database is used as the Relational Database Management System (RDBMS), and queries are written in standard Oracle SQL. The project is executed in a terminal-based environment such as Oracle SQL*Plus or Oracle Live SQL.

3. ER Model and ER Diagram

An entity is an object about which data is stored, such as a student, teacher, or course. Each entity possesses attributes that describe its properties. In this project, there are six main entities: Department, Teacher, Student, Course, Enrollment, and Result.

Dept_ID is the primary key attribute of Department. Student attributes include Student_ID (key), Student_Name, Gender, Date_Of_Birth, and Phone. Teacher includes Teacher_ID (key), Teacher_Name, Designation, Email, and Salary.

Department, Teacher, Student, and Course are strong entities because each has its own independent primary key. Enrollment and Result act as relationship tables connecting students and courses. A department has many teachers and many students (one-to-many relationships). A teacher teaches one or more courses, and a student can enroll in multiple courses, creating many-to-many relationships resolved via Enrollment and Result.

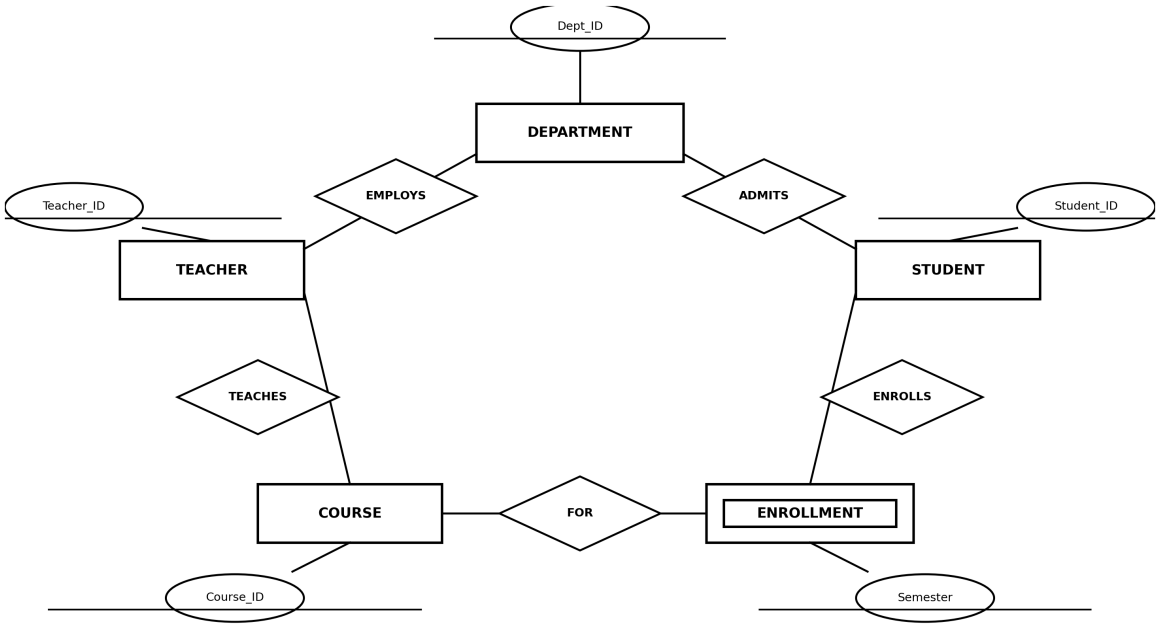


Figure 3.1: ER Diagram of the University Student Management System. Rectangles represent entities, diamonds represent relationships, ovals represent key attributes, and the double rectangle represents the weak entity.

4. ER-to-Relational Mapping and Table Description

The ER model is converted into relational tables following standard database mapping rules. Strong entities become tables with their key attributes acting as primary keys. Foreign keys are placed on the 'many' side of relationships to maintain referential integrity.

4.1 Relational schema

DEPARTMENT (Dept_ID, Dept_Name, Head_Name, Established_Year)
TEACHER (Teacher_ID, Teacher_Name, Designation, Dept_ID, Email, Salary)
STUDENT (Student_ID, Student_Name, Gender, Date_Of_Birth, Phone, Dept_ID, Admission_Year)
COURSE (Course_ID, Course_Name, Credits, Dept_ID, Teacher_ID)
ENROLLMENT (Student_ID, Course_ID, Semester, Enrollment_Date)
RESULT (Student_ID, Course_ID, Marks, Grade, Exam_Date)

4.2 Description of the tables

Table 4.1: Description of the six tables

Table	Primary Key	Foreign Key and Purpose
Department	Dept_ID	Stores department name, head, and establishment year.
Teacher	Teacher_ID	Dept_ID refers to Department. Stores teacher designation and salary.
Student	Student_ID	Dept_ID refers to Department. Stores student personal and contact details.
Course	Course_ID	Dept_ID & Teacher_ID refer to Department & Teacher. Stores course credits.
Enrollment	Student_ID + Course_ID	Refers to Student and Course. Stores semester and registration date.
Result	Student_ID + Course_ID	Refers to Student and Course. Stores marks, grades, and exam date.

5. DDL - Table Management

Data Definition Language (DDL) commands are used to define and modify database table structures. Data types such as NUMBER, VARCHAR2, CHAR, and DATE are utilized alongside constraints like PRIMARY KEY, UNIQUE, NOT NULL, and CHECK.

```
CREATE TABLE Department (
  Dept_ID NUMBER(4) PRIMARY KEY, Dept_Name VARCHAR2(50) NOT NULL UNIQUE,
  Head_Name VARCHAR2(60), Established_Year NUMBER(4) CHECK (Established_Year > 1900)
);

CREATE TABLE Teacher (
  Teacher_ID NUMBER(4) PRIMARY KEY, Teacher_Name VARCHAR2(60) NOT NULL,
  Designation VARCHAR2(30) CHECK (Designation IN ('Professor','Associate
Professor','Assistant Professor','Lecturer')),
  Dept_ID NUMBER(4) REFERENCES Department(Dept_ID), Email VARCHAR2(60) UNIQUE, Salary
NUMBER(8,2) CHECK (Salary > 0)
);

CREATE TABLE Student (
  Student_ID NUMBER(5) PRIMARY KEY, Student_Name VARCHAR2(60) NOT NULL,
  Gender CHAR(1) CHECK (Gender IN ('M','F')), Date_Of_Birth DATE, Phone VARCHAR2(15) NOT
NULL,
  Dept_ID NUMBER(4) REFERENCES Department(Dept_ID), Admission_Year NUMBER(4) CHECK
(Admission_Year BETWEEN 2000 AND 2030)
);

CREATE TABLE Course (
  Course_ID VARCHAR2(10) PRIMARY KEY, Course_Name VARCHAR2(60) NOT NULL,
  Credits NUMBER(2) CHECK (Credits BETWEEN 1 AND 4), Dept_ID NUMBER(4) REFERENCES
Department(Dept_ID),
  Teacher_ID NUMBER(4) REFERENCES Teacher(Teacher_ID)
);

CREATE TABLE Enrollment (
  Student_ID NUMBER(5) REFERENCES Student(Student_ID), Course_ID VARCHAR2(10) REFERENCES
Course(Course_ID),
  Semester VARCHAR2(20) NOT NULL, Enrollment_Date DATE, PRIMARY KEY (Student_ID, Course_ID)
);

CREATE TABLE Result (
  Student_ID NUMBER(5) REFERENCES Student(Student_ID), Course_ID VARCHAR2(10) REFERENCES
Course(Course_ID),
  Marks NUMBER(3) CHECK (Marks BETWEEN 0 AND 100), Grade VARCHAR2(2), Exam_Date DATE,
PRIMARY KEY (Student_ID, Course_ID)
);
```

5.1 DESCRIBE and ALTER TABLE

Table structures can be inspected using DESCRIBE and modified via ALTER TABLE statements:

```
ALTER TABLE Student ADD Email VARCHAR2(60);
ALTER TABLE Teacher MODIFY Salary NUMBER(10,2);
ALTER TABLE Course RENAME COLUMN Credits TO Credit_Hours;
ALTER TABLE Student DROP COLUMN Email;
```

6. DML - Data Manipulation

Data Manipulation Language (DML) commands interact with table data using INSERT, UPDATE, and DELETE. Dates are converted using TO_DATE('DD-MM-YYYY'). Sample insertion statements are shown below:

```
INSERT INTO Department VALUES (101, 'Computer Science and Engineering', 'Dr. Anwar
Hossain', 1995);
INSERT INTO Teacher VALUES (1, 'Mobarak Hossain Al-Mamon', 'Lecturer', 101,
'mobarak@diu.edu.bd', 55000);
INSERT INTO Student VALUES (21, 'Md Rayhan Miah', 'M', TO_DATE('15-03-2004','DD-MM-YYYY'),
'01745532902', 101, 2024);
INSERT INTO Student VALUES (22, 'Rafi Hasan', 'M', TO_DATE('10-01-2004','DD-MM-YYYY'),
'01711112222', 101, 2024);
INSERT INTO Course VALUES ('CSE-301', 'Database Management System', 3, 101, 1);
INSERT INTO Enrollment VALUES (21, 'CSE-301', 'Fall 2026',
TO_DATE('01-07-2026','DD-MM-YYYY'));
INSERT INTO Result VALUES (21, 'CSE-301', 85, 'A', TO_DATE('08-11-2026','DD-MM-YYYY'));
```

6.1 UPDATE and DELETE

```
UPDATE Teacher SET Salary = 60000 WHERE Teacher_ID = 1;
UPDATE Result SET Grade = 'A' WHERE Student_ID = 21 AND Course_ID = 'CSE-301';
DELETE FROM Enrollment WHERE Student_ID = 28 AND Course_ID = 'EEE-201';
```

6.2 Oracle and MySQL syntax comparison

Operation	**Oracle**	**MySQL**
Text Data Type	VARCHAR2(50)	VARCHAR(50)
Number Data Type	NUMBER(4)	INT
Text to Date	TO_DATE('08-11-2026', 'DD-MM-YYYY')	STR_TO_DATE('08-11-2026', '%d-%m-%Y')
Replacing NULL	NVL(column, 0)	IFNULL(column, 0)
Limiting Rows	WHERE ROWNUM <= 5	LIMIT 5

7. Sample Queries and Output

Important queries implemented in the project along with their terminal outputs:

Query 1: Show ID, name, gender and phone of all students.

```
SELECT Student_ID, Student_Name, Gender, Phone FROM Student;
```

STUDENT_ID	STUDENT_NAME	GENDER	PHONE
21	Md Rayhan Miah	M	01745532902
22	Rafi Hasan	M	01711112222
23	Maria Hasan	F	01722223333
24	Tahzib Rahman	M	01733334444

Query 2: Show each department ID only once using DISTINCT.

```
SELECT DISTINCT Dept_ID FROM Student;
```

DEPT_ID
101
102
103
104

Query 3: Show total marks out of 300 for student ID 21 (calculated field & alias).

```
SELECT Student_ID, (85+78+82) AS Total_Marks FROM Result WHERE Student_ID = 21;
```

STUDENT_ID	TOTAL_MARKS
21	245

Query 4: Join student name with course name and marks.

```
SELECT S.Student_Name, C.Course_Name, R.Marks, R.Grade FROM Student S, Course C, Result R
WHERE S.Student_ID = R.Student_ID AND C.Course_ID = R.Course_ID;
```

STUDENT_NAME	COURSE_NAME	MARKS	GRADE
Md Rayhan Miah	Database Management System	85	A
Md Rayhan Miah	Data Communication	78	B+
Rafi Hasan	Database Management System	72	B
Maria Hasan	Database Management System	90	A+

8. Conclusion

In this project, a comprehensive database for a University Student Management System has been successfully designed and implemented using Oracle SQL. Starting from the conceptual ER model, the schema was systematically mapped into six relational tables: Department, Teacher, Student, Course, Enrollment, and Result.

DDL commands were employed to construct table structures and enforce rigorous integrity constraints, while DML commands managed data insertion, updates, and deletions. Through powerful SELECT queries involving filtering, sorting, aggregate functions, GROUP BY, HAVING clauses, subqueries, and multi-table joins, practical administrative tasks—such as computing total marks, tracking department enrollments, and evaluating student performance—were efficiently accomplished.

While the system effectively meets all Database Management System Lab objectives, it has certain limitations: it operates entirely via terminal-based SQL execution without a graphical user interface, and it does not handle fee billing or attendance tracking. In future iterations, a web or desktop application front-end can be integrated, and additional modules for attendance and financial accounts can be incorporated.

Working on this project provided a thorough, practical understanding of database design principles, ER diagram translation, and structured query formulation for real-world information systems.